

종이 장부, 클라우드 원장

CSV는 **종이 장부**다 — 내가 쓰고, 파일 캐비닛(GitHub)에 넣어두고, 필요하면 다시 꺼내 읽는다. DB(Supabase)는 **클라우드 원장**이다 — 나도 쓰고 다른 지점(meritz)도 동시에 같은 장부를 본다. 지금 이 앱은 이 둘을 용도에 맞게 나눠 쓰고 있고, 그 경계가 왜 거기 그어져 있는지, 어제·오늘 겪은 버그들이 왜 하필 그 경계 근처에서만 터지는지를 이 리포트에서 그림과 함께 정리한다.

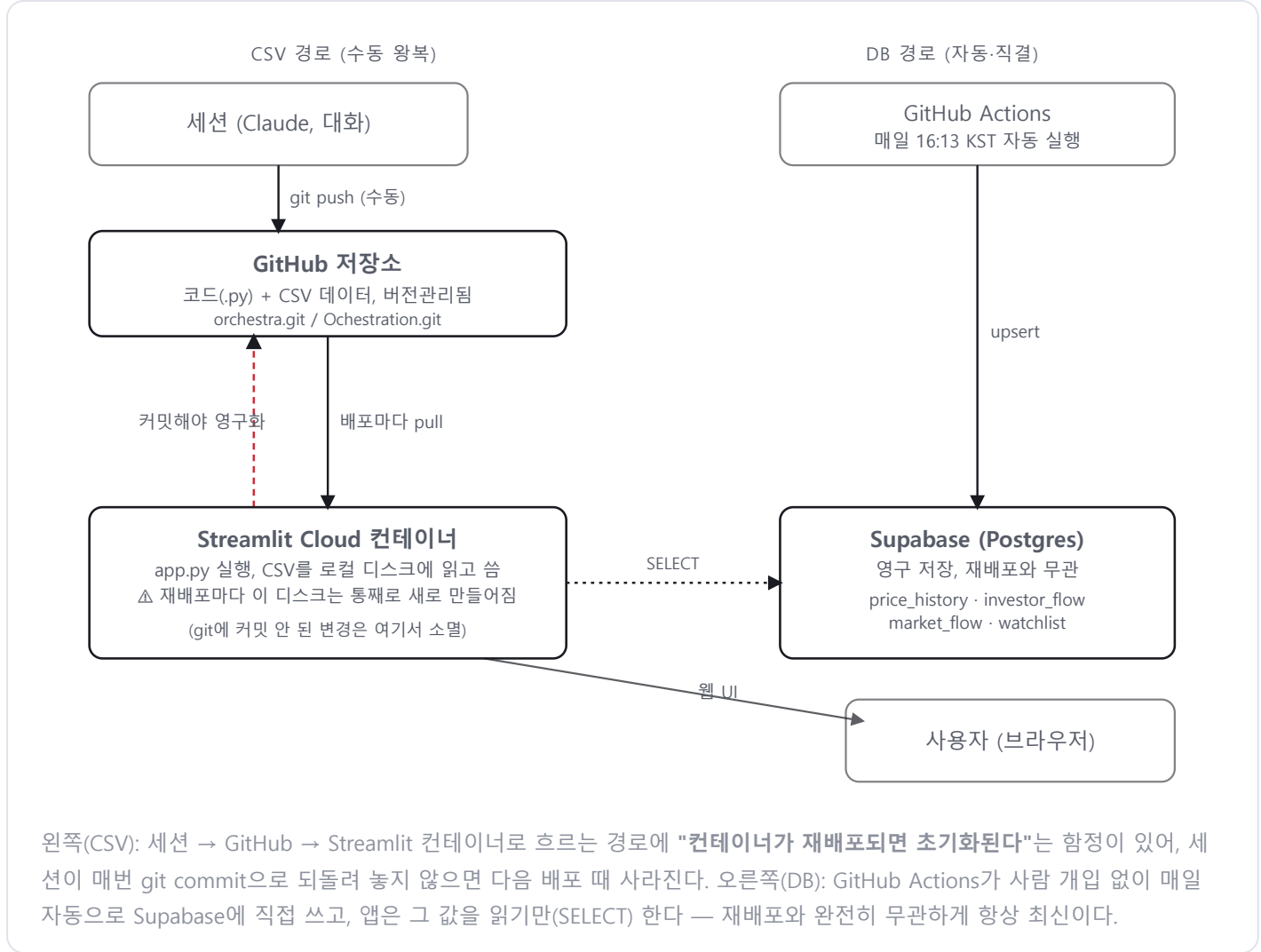
작성 2026-09-15 대상: new1(Orchestra) · meritz(Orchestration)

한 줄 요약: 코드가 아니라 "같은 사실을 누가 어디서 몇 번 저장하는가"가 지금까지 터진 버그의 90%를 설명한다.

혼자만 아는 사실(내 거래 내역)은 CSV+git으로 충분하고 실제로 잘 작동해왔다. 반대로 **여러 곳(153종목 스크리너, new1·meritz 양쪽 앱)이 동시에 참조해야 하는 사실** (코스피 시세, 외국인 수급)은 CSV로 관리하는 순간 "누가 언제 받아왔느냐"에 따라 서로 다른 버전이 생긴다 — 어제의 investor_flow 스크레이핑 중단, 오늘의 new1·meritz 코스피 수치 불일치가 전부 이 구조적 이유 하나로 설명된다.

01 지금 실제로 이렇게 돌아간다

말로 설명하면 복잡한데, 그림으로 보면 왜 CSV 쪽(왼쪽)이 DB 쪽(오른쪽)보다 손이 더 많이 가는 지 한눈에 보인다.



02 CSV가 담당하는 것, DB가 담당하는 것

기준	CSV (+ git)	DB (Supabase)
담당 데이터	내 거래·보유·현금·캐시(종목코드·섹터)·히스토리 스냅샷	153+ 관심종목의 시세·거래량·외국인수급 (price_history 등)
누가 읽나	new1 혼자, meritiz 혼자 (서로 다른 파일)	new1·meritiz 둘 다 (같은 테이블)
누가 쓰나	세션이 실행하는 스크립트(ingest_daily.py) 1개, 한 번에 하나씩	GitHub Actions cron(매일 1회) + 세션이 가끔 직접
규모	거래 수백 건, 파일 몇~수십 KB	153종목×수십 일 = 수천~수만 행
영구성	git 커밋이 곧 백업(무료 버전관리 포함)	DB 자체가 영구(클라우드 서버에 계속 존재)
동시 접근	필요 없음(한 번에 한 세션만 씀)	필요함(cron 쓰기 + 앱 두 개가 동시에 읽음)
조회 방식	매번 전체를 pandas로 읽어서 메모리에서 계산	필요한 행만 SQL로 필터링해서 가져옴

03 DB 없이 CSV만으로 굴러가는 이유

"DB가 없으면 원래 앱이 안 돌아가는 거 아니야?"라고 생각하기 쉬운데, 이 앱의 CSV 파트는 DB가 아예 없어도 처음부터 지금까지 문제없이 돌아가고 있다. 이유는 네 가지 전제가 전부 충족되기 때문이다:

1. 데이터가 작다

거래가 수백 건, 보유종목이 수십 개 — pandas가 CSV 전체를 통째로 메모리에 읽어도 밀리초 단위로 끝난다. DB의 "필요한 행만 조회"라는 장점 자체가 발휘될 규모가 아니다.

2. 한 번에 한 사람(세션)만 쓴다

DB가 해결하는 핵심 문제는 "여러 명이 동시에 같은 행을 고치면 어떻게 하나"인데, 이 CSV들은 세션이 순서대로 하나씩 스크립트를 돌려서 고치지 두 프로세스가 동시에 같은 파일에 쓰는 경우가 없다. 동시성 문제가 원천적으로 발생하지 않는다.

3. 버전관리를 git이 공짜로 준다

"어제 이 값이 뭐였지"를 알고 싶으면 `git log / git show` 로 바로 볼 수 있다 — DB였다면 이런 이력 추적을 위해 별도로 감사로그(audit log) 테이블을 설계해야 했을 것이다. CSV+git 조합이 오히려 이 앱의 "매일 무슨 일이 있었는지 기록한다"는 목적에 더 잘 맞는다.

4. 복잡한 질의가 필요 없다

"이 종목의 평단가"처럼 계산은 복잡해도(\$1-1 재생 로직), 그 계산에 필요한 건 **내 거래 전체** 뿐이다 — "여러 테이블을 조인해서" 같은 진짜 DB가 필요한 질의 패턴이 없다. Fishing/Link처럼 "153종목 중 조건에 맞는 것만" 걸러내는 질의가 나오는 순간부터 DB가 필요해졌고, 실제로 그 부분만 \$6-9에서 DB로 옮겨졌다.

04 파이썬 파일이 이렇게 나뉜 이유

파일	줄 수	역할
app.py	493	진입점 — 로그인, 데이터 로드, 새로고침, 탭 조립만
portfolio_core.py	3,011	데이터 계층 전체(로드/저장/재생/시세조회/CSV파싱/지표계산) — UI가 없는 순수 로직 이라 테스트하기 쉽고, ingest_daily.py와도 공유
ui_portfolio_tab.py	1,270	포트폴리오 탭 렌더링만
ui_transactions_tab.py	907	거래 기록 탭 렌더링만
ingest_daily.py	176	매매일지 CSV 반영 스크립트(앱과 무관하게 단독 실행)
db_fetch_daily_prices.py	166	GitHub Actions가 매일 돌리는 cron 스크립트
tests/test_portfolio_core.py	1,560	회귀 테스트 87개

meritz는 app.py(1,910줄)+portfolio_core.py(1,765줄) 두 파일에 다 들어있다 — UI 분리를 아직 안 했다(\$2, "필요해지면 다시 검토"로 남겨둔 상태). new1이 먼저 커지면서 꼬인 거라 meritz는 아직 그 시점이 안 왔다고 보면 된다.

기준은 단순하다 — **"UI를 그리는 코드"**(streamlit 위젯 호출)와 **"데이터를 계산하는 코드"**(pandas만 다루는 순수 함수)를 분리했다. 이렇게 나눈 이유: (1) portfolio_core.py는 화면이 없는 순수 로

직이라 `tests/` 에서 Streamlit 없이도 테스트할 수 있다. (2) `ingest_daily.py`(매일 실행되는 배치 스크립트)와 `app.py`(웹 화면)가 **같은 계산 로직을 공유**해야 하는데, 그 공유 지점이 바로 `portfolio_core.py`다 — 로직이 두 벌로 복제되면 한쪽만 고치고 다른 쪽을 잇는 사고가 난다(실제로 이런 종류의 버그를 §1-1 원칙으로 막고 있음).

05 세션(나)이 실제로 어떻게 고치는지

매번 파이썬 파일 7,000줄을 전부 읽지는 않는다. 오늘 실제로 있었던 사례로 설명하면 —

사례: "LINK에서 날짜가 이상하게 뜬다"는 신고

1) 신고 내용에 있는 단어("기준일", "경과일")로 `ui_portfolio_tab.py` 안에서 `grep` — 두 단어가 같이 쓰인 줄을 바로 찾음(1,270줄짜리 파일에서 처음부터 안 읽고 검색으로 직행). 2) 그 줄 앞뒤 30줄만 Read로 열어서 문맥 확인 — `link_watch_status` 가 "플래그일" 기준으로 경과일을 계산하는데 라벨은 "기준일"이라는 다른 필드를 잘못 쓰고 있는 걸 발견. 3) 한 줄만 수정(기준일 → 플래그일). 4) `pytest` 로 회귀 테스트 85개를 돌려서 이 수정이 다른 곳을 안 건드렸는지 확인. 5) `git commit`.

이 흐름이 가능한 이유는 두 가지다 — **파일이 역할별로 나뉘어 있어서** "어디를 찾아야 할지"가 좁혀지고(UI 문제면 `ui_*.py`, 계산 문제면 `portfolio_core.py`), **회귀 테스트가 안전망 역할**을 해서 "내가 고친 게 다른 걸 깨뜨렸는지"를 직접 코드를 다 안 읽어도 자동으로 확인해준다. 반대로 오늘 9/11 `asset_history` 버그처럼 **기존 테스트가 없던 영역**은 실제로 실행해서 숫자를 대조해보는 수밖에 없었다 — 이게 "테스트가 있는 곳은 빠르게, 없는 곳은 검증해가며 천천히" 라는 실제 작업 리듬이다.

06 GitHub · Streamlit Cloud · Supabase — 사실 이미 셋 다 "클라우드"다

"깃허브에서 클라우드로 언제 가나"라는 질문은 사실 이미 답이 반쯤 나와 있다 — **이 앱은 이미 세 개의 클라우드 서비스를 쓰고 있다.** 헷갈리는 건 세 서비스가 각자 다른 일을 하는데 이름이 비슷하게 느껴져서다:

- **GitHub** = 코드와 (지금은) CSV 데이터의 **버전관리 저장소**. "회사 서버"가 아니라 "파일 캐비닛" 역할 — 여기 있는 파일이 저절로 실행되진 않는다.
- **Streamlit Community Cloud** = 그 GitHub 저장소를 당겨와서 `app.py` 를 실제로 **실행하는 서버**. 사용자가 보는 웹페이지가 여기서 나온다. 이미 클라우드다 — 다만 컨테이너가 재배포마다 새로 만들어지는 "휘발성" 서버라는 게 §1-5의 근본 원인.
- **Supabase** = Postgres 데이터베이스를 호스팅하는 **클라우드 DB 서비스**. 이미 이 앱의 일부(153종목 시세)가 여기 저장돼 있다.

즉 "GitHub에서 클라우드로 옮긴다"는 표현은 정확하지 않다 — 앱은 이미 클라우드에서 돌고 있고, 데이터의 일부(시세)도 이미 클라우드 DB에 있다. 진짜 남은 질문은 "내 거래·보유 데이터(지금 GitHub 저장소 안 CSV 파일)를 언제 Supabase 같은 클라우드 DB로 옮기나"다 — 어제·오늘 계속 얘기한 그 마이그레이션이 바로 이거고, GitHub나 Streamlit Cloud 자체를 떠나는 일은 아니다(코드는 계속 GitHub에 있고 앱도 계속 Streamlit Cloud에서 돈다).

07 그래서, 언제 옮기나 — 어제와 오늘의 판단

어제(9/14) 이 질문에 **시급도 10~20/100**으로 답했다 — 실제 데이터 유실은 없었고, lock-step류 버그는 매번 그날 안에 잡혔기 때문. 오늘(9/15) 같은 부류의 버그가 재발했다 — new1·meritz가 **같은 코스피 값을 서로 다르게 기억하는 걸 발견**(\$방금 리포트). 그래서 오늘 **35~40/100**으로 올렸다. 60~70까지 안 올린 이유는 이번 영향이 작아서다(0.06% 오차, 실제 판단을 왜곡할 정도는 아님) — 하지만 패턴 자체가 재발했다는 사실은 그대로 기록해 둔다.

다음에 시급도를 60~70으로 더 올릴 조건

1. — 같은 부류 버그가 또 재발하거나(3번째), `check_history_alignment()` 같은 기존 방어막을 뚫고 발생.
2. — 실제로 데이터가 유실되거나, 틀린 채로 하루 이상 방치되는 사고.
3. — 어느 한쪽이라도 발생하면 로드맵대로 **index_history/bigcap_history**부터 (이미 Supabase price_history와 같은 시계열 구조라 옮기기 제일 쉽고, 오늘 문제가 난 바로 그 데이터이기도 함) 먼저 Supabase 공유 테이블로 옮기는 걸 시작한다 — transactions.csv처럼 재생(replay) 로직이 얹힌 무거운 데이터는 그 다음.

결론: 지금 구조가 "빨리 만들어져서 구멍이 많다"기보다는, "혼자 쓰던 장부가 여러 지점이 같이 보는 장부로 요구사항이 바뀌는 중"인 자연스러운 성장통에 가깝다. 각 기능은 그때그때 테스트와 문서화를 곁들여 만들어졌지만(\$1~\$6-28 전부), "공유돼야 할 사실을 두 곳에 따로 저장한다"는 설계 선택 하나가 이번 달 재발한 버그 대부분의 공통 조상이다 — 이 리포트의 구조도가 보여주는 딱 그 지점.